

ICLR 2024 | 别再事后证明了：把表达力「写」进图神经网络

法国 LITIS 与 LIFAT 实验室提出 G2N2:用上下文无关文法从矩阵语言直接推导出可证明 3-WL 的图神经网络

衡量一个图神经网络 (GNN) 到底有多「聪明」, 业界最常用的标尺是 Weisfeiler-Lehman (WL) 层级: 如果两张不同构的图连 k 阶 WL 测试都分不开, 那么再强的 GNN 也分不开。于是多年来的惯例是, 先凭直觉设计一个模型, 再回过头去证明它能匹配某一阶 WL 测试。表达力的保证, 总是作为「事后」的一纸证明补上来的。

问题在于, 这条路没有通用配方。每设计一个新架构, 就要重新写一遍表达力证明; 而反过来的方向, 也就是「给定一个已知表达力的数学片段, 机械地造出一个继承其表达力的网络」, 一直没有系统方法。已有的 3-WL 模型也各有短板: 3-IGN 的算子基规模随 $2k$ 阶 Bell 数爆炸, 内存吃紧; PPGN 虽然可算, 却只被证明能模拟 2-FWL, 与语言表达力之间没有构造性的桥梁。

ICLR 2024 上的这篇 G2N2 反其道而行: 它不再「先设计后证明」, 而是「先选定一个表达力已知的语言片段, 再把它的规则直接读成网络的层」。整条流水线是自动的, 产出的网络可证明达到 3-WL 表达力, 而且在实验里比现有 3-WL 模型更快、更准。下图是全文的一张总览。

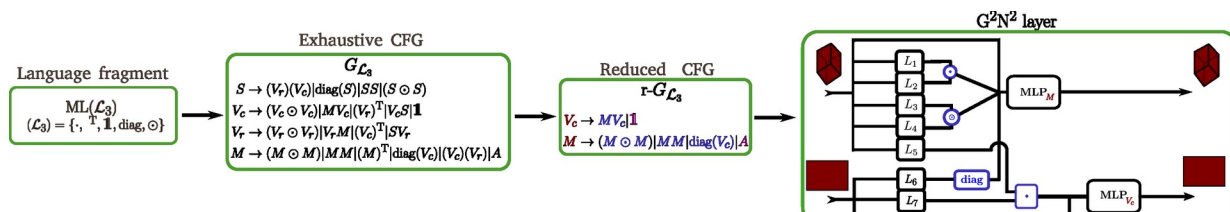


图 1 G2N2 的整条推导流水线。从与 3-WL 等价的矩阵语言片段 $ML(L_3)$ 出发, 先写出穷尽的上下文无关文法 $G(L_3)$, 再约简为紧凑的 $r-G(L_3)$, 最后把约简文法里存活下来的变量当作网络输入、产生式规则当作层内运算, 直接得到 G2N2 的一层。

论文: <https://arxiv.org/abs/2303.01590> 代码: <https://github.com/JPiquenot/Weisfeiler-and-Lehman-go-grammatical>

背景:从 WL 层级到矩阵语言

这套方法的起点, 是一个漂亮的观察: 1-WL 与 3-WL 测试都可以改写成一种矩阵语言 MATLANG 的片段。具体地说, 两张图在 3-WL 看来一样, 当且仅当片段 $ML(L_3)$ 里你能写出的每一个「句子」在两张图上取值都相同。这在组合数学(WL 测试)和线性代数(矩阵运算)之间架起了一座桥。

但桥不等于路。把这样一个片段变成真正可训练的网络, 此前只能一个个手工来做, 得到的模型往往拿不到完整的 3-WL 保证。例如同样从 MATLANG 视角出发的 GNNML3, 只被证明比 1-WL

更强,而没能证到 3-WL,原因恰恰是缺少一条系统化的推导路径。G2N2 要做的,就是把这条路一次性铺好。

第一步:从语言到文法,再到约简文法

推导从一组算子开始: $L3 = \{ \cdot \text{ (矩阵乘)}, ^T \text{ (转置)}, 1 \text{ (全一向量)}, \text{diag} \text{ (取对角)}, \odot \text{ (逐元素乘)} \}$ 。作者先写出一部穷尽的上下文无关文法 $G(L3)$,它生成的句子恰好就是片段 $ML(L3)$ 。这一步把「一个语言」变成了「一组产生式规则」。

接着是关键约简:在保证每一步都不损失 3-WL 表达力的前提下,不断删去冗余的规则和变量,直到只剩下最精简的核心产生式 $r-G(L3)$ 。约简既让后续的网络更轻,也顺带暴露出「哪些运算真正重要」。要理解规则如何变成计算,可以看下面这张一次迭代的示意图。

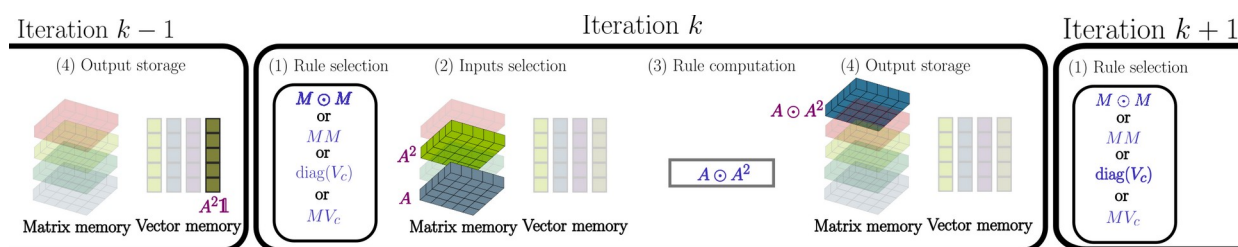


图 2 一次迭代的四个步骤。(1) 规则选择:从 $(M \odot M)$ 、 (MM) 、 $\text{diag}(V_c)$ 、 (MV_c) 中挑一条产生式;(2) 输入选择:按所选规则从矩阵记忆和向量记忆里取出对应的输入;(3) 规则计算:执行该运算(图中为 $A \odot A^2$);(4) 输出存储:把结果写回相应的记忆。逐次迭代,记忆里就积累出越来越丰富的图特征。

第二步:把文法规则读成网络的层

约简文法一旦到手,网络几乎是「读」出来的:存活下来的变量告诉你网络的输入该是什么(一组刻画节点与边的特征张量),而每一条存活的产生式规则,都对应层里的一块运算。这正是 G2N2(Grammatical Graph Neural Network)名字的由来。

一层内部维护两份记忆:边(矩阵)记忆 C 和节点(向量)记忆 H 。可学习的线性块 $L1$ 到 $L7$ 负责组合张量的各个切片,约简后的规则 $(M \odot M)$ 、 (MM) 、 $\text{diag}(V_c)$ 、 (MV_c) 在其上计算,再由两个多层感知机 MLP_M 与 MLP_{Vc} 把结果拼接、融合,分别输出下一层的 C 和 H 。下图把这一层的数据流拆开来看。

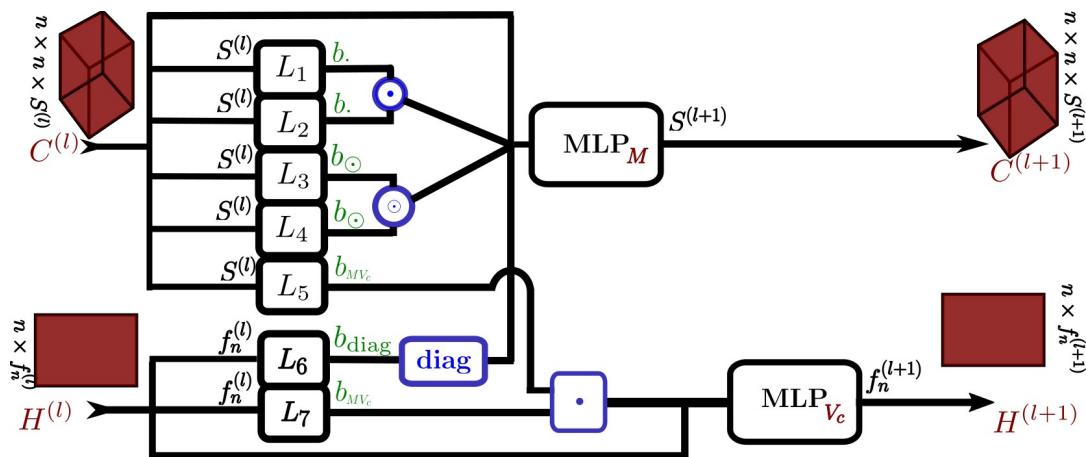


图 4 G2N2 一层的内部结构。左侧输入边张量 C 与节点张量 H ; $L1-L5$ 线性组合 C 的切片并按规则 (MM)、 $(M \odot M)$ 做两两乘积, $L6-L7$ 处理 H 并经 diag 化为对角矩阵; 这些中间量连同输入一起送入 MLP_M 得到新的边张量 $C^{(l+1)}$, 送入 MLP_{Vc} 得到新的节点张量 $H^{(l+1)}$ 。

把这样的层堆叠 k 次, 再在最后接上排列等变(permutation-equivariant)的读出函数, 一个完整的 G2N2 就成形了。读出分别作用在节点记忆 H 、以及边记忆 C 的对角与非对角元素上, 最后交给决策层给出预测。整体架构如下。

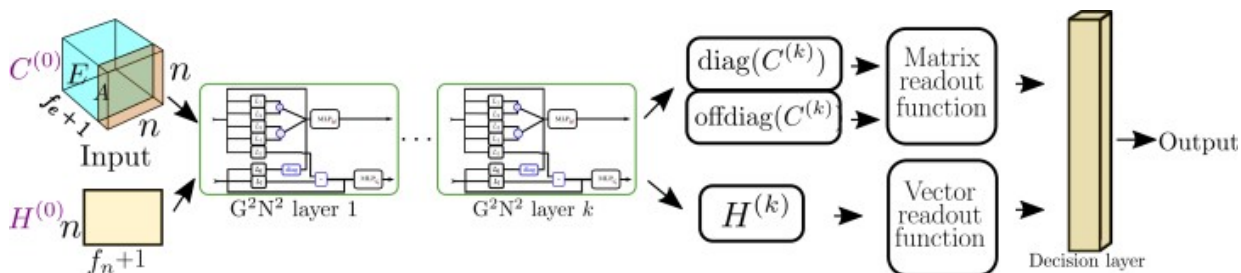


图 5 G2N2 从图到输出的完整架构。输入的点与边嵌入依次经过 k 个 G2N2 层不断更新; 随后对 $H^{(k)}$ 、以及 $C^{(k)}$ 的对角与非对角部分分别做读出, 汇入决策层输出最终预测。

效果:QM9 分子回归

理论漂亮之外, G2N2 在实验上也站得住脚。第一个战场是 QM9: 约 130K 个小分子、12 个量子化学回归目标, 其中 R^2 是公认最难预测的一项。逐个目标单独训练时, G2N2 在全部 12 个目标上都取得了最低的 MAE, 同时每个 epoch 的训练还更快(98 秒对 PPGN 的 129 秒)。最戏剧性的是 R^2 目标: 误差从 PPGN 的 3.78 降到 0.342, 约 11 倍的改善; 而当 12 个目标一起训练时, 两者的差距还会进一步拉大(16.07 对 1.19), 可见约束越强, G2N2 的表达力优势越明显。

表 1 QM9 单目标回归的部分结果(指标为 MAE, 越低越好), 以及每 epoch 平均耗时。完整 12 个目标见论文表 4。

目标	PPGN	G2N2
μ	0.0934	0.0703
α	0.318	0.127
R^2	3.78	0.342
U0	0.022	0.0169

Cv	0.144	0.0429
每 epoch 耗时	129 s	98 s

分类与一次频谱压力测试

第二个战场是经典的 TUD 图分类基准,从 MUTAG、PTC 这类分子图到 IMDB 这类社交图。G2N2 在其中 5 个数据集上都优于第二名的 GNN 竞争者,并在 MUTAG、PTC、Proteins 上直接排名第一。

表 2 TUD 数据集上 G2N2 与最强 GNN 竞争者的对比(指标为准确率,越高越好;括号内为 G2N2 在 GNN 中的排名)。完整结果见论文表 6。

数据集	G2N2	排名	最强 GNN 竞争者
MUTAG	92.5±5.5	1	92.2±7.5
PTC	72.3±6.3	1	68.2±7.2
Proteins	80.1±3.7	1	77.4±4.9
NCI1	82.8±0.9	3	83.5±2.0
IMDB-B	76.8±2.8	2	77.8±3.3
IMDB-M	54.0±2.9	2	54.3±3.3

第三个战场更像一次「压力测试」:在 900 节点大图上学习频谱滤波器。已知消息传递网络天然偏向低通滤波,而高通、带通滤波才是真正考验表达力的地方。结果颇能说明问题:PPGN 在低通、高通上都接近满分,却在带通上崩溃(R^2 仅 0.1041);G2N2 则在三种滤波器上都保持了高分,带通 R^2 达到 0.8206。作者指出,PPGN 之所以在带通上失手,正是因为它缺少 G2N2 那份能显式保留边信息的记忆。

表 3 频谱节点回归上的 R^2 (10 次运行中位数,越高越好)。PPGN 在带通滤波上失效,G2N2 三类滤波器均表现稳健。

方法	低通	高通	带通
PPGN	0.9991	0.9925	0.1041
G2N2	0.9996	0.9994	0.8206

哪些规则真的重要:文法约简消融

最能体现这套方法特色的,是围绕文法约简做的消融实验。作者在 QM9 的 R^2 目标上,比较了完整文法 G(L3)、中间文法 i-G(L3) 与约简文法 r-G(L3):三者误差基本持平,说明约简只是去掉了冗余,没有动到表达力。但一旦越过约简文法、继续删掉核心规则,误差就会明显上升。

这种「退化」本身就是有用的信息:它量化了每条运算规则对性能的贡献。换句话说,当某个任务并不需要完整的 3-WL 表达力时,你可以照着这张图有依据地对模型做裁剪,而不是盲目地砍参数。

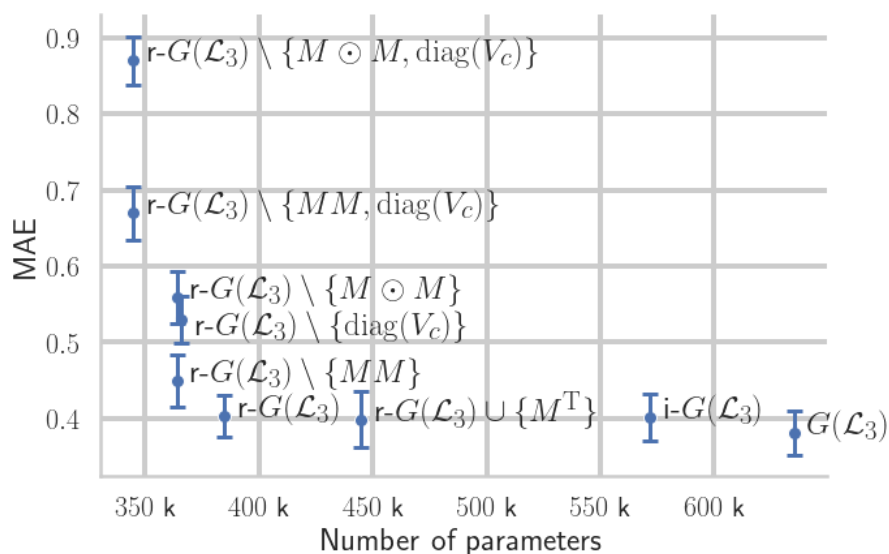


图 6 QM9 R^2 上参数量与 MAE 的散点。完整文法 $G(\mathcal{L}_3)$ 、中间文法 $i-G(\mathcal{L}_3)$ 、约简文法 $r-G(\mathcal{L}_3)$ 及其加入一条规则 (M^T) 的变体误差相近;而从 $r-G(\mathcal{L}_3)$ 中继续删去 $\{MM\}$ 、 $\{M \odot M\}$ 、 $\text{diag}(V_c)$ 等核心规则(记作 $\setminus$)后,MAE 明显抬高,验证了每条规则的作用。

小结:让表达力「与生俱来」

G2N2 真正改变的是 workflow:不再是先设计网络、再祈祷能证明它足够强,而是从一个表达力已知的语言出发,约简成干净的文法,然后把网络直接从规则里读出来,表达力在构造时就有了保证。落到实处,就是一个可证明 3-WL、同时比前辈更快更准的模型。

更值得注意的是,这套「文法化」框架本身是通用的:原则上,任何一个代数语言的片段,都能沿同样的路径被转成一个继承其表达力的网络。G2N2 是这个想法的第一个具体产物,而它指向的方向,是一整类「按需定制表达力」的图神经网络设计方法。